

Panasonic Lumix GX880 Wi-Fi `accctrl` Command Technical Overview

Purpose and Function of the `accctrl` Command

In Panasonic Lumix Wi-Fi control protocol, the `accctrl` command handles **access control and pairing** between the camera and a remote device (smartphone, tablet, or computer). It is essentially the handshake step that **authorizes a device for remote control**. When a device first attempts to connect via Wi-Fi, the camera uses `accctrl` to verify and register that device. This ensures that only approved devices can control the camera. In other words, `accctrl` manages the initial pairing and permission granting process for the Lumix's wireless remote control function ¹. Once the handshake is successful, the camera enters *remote control mode* (often displaying "Under Remote Control" on its screen) and accepts further commands from the device.

Request URL Format and Required Parameters

The `accctrl` command is invoked via an HTTP GET request to the camera's built-in web server (typically at the camera's IP address on its Wi-Fi network). The basic format is:

```
http://<camera-ip>/cam.cgi?mode=accctrl&type=req_acc&value=<device-id>&value2=<device-name>
```

Parameters:

- `mode=accctrl` – indicates the access control mode.
- `type=req_acc` – indicates a *request for access* (pairing request).
- `value` – a unique identifier for the device (explained in detail below).
- `value2` – the name of the device (human-readable name).

For example, a real pairing request captured from a Lumix GX80 looks like this (as used in a user script):

```
GET http://192.168.54.1/cam.cgi?
mode=accctrl&type=req_acc&value=4D454930-0100-1000-8001-024500021C98&value2=SM-
G903
```

Here, `value=4D454930-0100-1000-8001-024500021C98` and `value2=SM-G903` were sent to the camera ². Another example from a GX80 reverse engineering project uses `value=4D454930-0100-1000-8001-02FA000430C6` with `value2=MI%205` (device name "MI 5") ¹. These examples confirm the URL structure and parameters: `mode=accctrl`, `type=req_acc`, and the

two device-identifying values. The requests are typically sent as HTTP GET with query parameters (sometimes using `curl -G` as shown in community scripts ²).

Role of `value` and `value2` (Device ID and Name)

`value` (**Device ID**): This is a unique identifier for the connecting device. In practice it is a 128-bit GUID/UUID string formatted with hyphens. For instance, `4D454930-0100-1000-8001-024500021C98` is a GUID used by a Samsung phone in an example ³ . The Lumix Image App (or custom controllers) generates this ID to represent the device. The camera uses this ID to recognize the device on future connections. Essentially, it serves as the *device's fingerprint* in the pairing process. The GUID may be randomly generated or derived from the device; in any case, it should remain constant for that device/app so that the camera can remember it.

`value2` (**Device Name**): This is a user-friendly name for the device. In the examples above, `SM-G903` or `MI 5` were sent as the device name ³ ⁴ . Commonly this might be the phone model or a name like "John's iPhone". The camera may display this name when asking for confirmation, so the user knows which device is trying to connect. It also likely gets stored in the camera's Wi-Fi history along with the GUID. The device name can be URL-encoded if it contains spaces or special characters (e.g. `MI%205` for "MI 5").

In summary, `value` is a unique device ID and `value2` is the device's name. Together, they identify the remote controller to the camera during the pairing handshake.

First-Time Connection Behavior (Pairing Process)

On the **first ever connection attempt** from a new device (i.e. the camera has not seen the `value` GUID before), the `accctrl` request triggers the camera to prompt the user for authorization. The camera's LCD will display a message confirming that a device is trying to connect and usually asks the user to accept or reject. For example, the camera might show "**Connect to smartphone <XYZ>? [Yes / No]**" or a similar prompt ⁵ . The device name sent in `value2` may be shown in this prompt (depending on camera model/UI). The user must physically acknowledge this on the camera (by pressing a button or tapping "Yes"). This is a security step to prevent unauthorized remote control.

During this time, the `accctrl&type=req_acc` HTTP request from the device is essentially held pending. The camera does not immediately return "ok"; it waits for the user's decision. If the user accepts on the camera, the pairing proceeds. If the user denies or no one accepts in time, the request will fail (typically the HTTP request will time out or the camera may return an error code after a timeout). In community testing, if the first attempt timed out (user didn't confirm in time), simply retrying the `accctrl` request after ensuring proper procedure would eventually succeed ⁶ .

When the user confirms, the camera finalizes the pairing: it registers that GUID as an authorized device. At that point the camera responds to the waiting request with a success message. According to documented behavior, the **success response** to the initial `accctrl` request is:

```
ok,<CameraID>,remote,encrypted
```

For example, on a GX80 the response was `"ok,GX80-468E59,remote,encrypted"` ⁷. Here `GX80-468E59` is the camera's identifier (the camera's SSID or unique ID), and the flags `remote,encrypted` indicate the mode. "remote" signifies the camera is in remote control mode, and "encrypted" indicates that the live view stream will be encrypted (Panasonic's apps use an encrypted video stream for live view) ⁷. After sending this response, the camera typically changes its state to fully enable remote control features (displaying "Under Remote Control" on the camera screen ⁷).

In summary, on first-time use the `accctrl` handshake behaves as follows:

- The app connects to the camera's Wi-Fi and issues the `mode=accctrl&type=req_acc` request with its ID and name.
- The camera detects an unknown ID and **prompts the user to allow access** ⁵.
- Upon user approval, the camera returns an **"ok,..."** **success response** to the app and enters remote control mode ⁷.
- If the user declines or no response is given, the handshake fails (no "ok" returned).

Notably, some scripts and documentation suggest that the **device name might be sent via a separate command either before or after accctrl**. In the official Panasonic Image App flow, the app sets the device name on the camera as well. For instance, community scripts sometimes call: `cam.cgi?mode=setsetting&type=device_name&value=<Name>` in addition to the `accctrl` request ⁴. This may ensure the camera has the proper name stored, but in many tests the `value2` parameter in the `accctrl` request is sufficient for the name. In practice, whether the name is set before or after, the camera will register the name along with the device once pairing is accepted.

During first-time pairing, the camera only needs to be authorized once. After the "ok" response, the device is now remembered as an authorized controller.

Behavior on Subsequent Connections (Previously Authorized Devices)

Once a device has been paired via a successful `accctrl` handshake, subsequent connections become seamless. The camera **recognizes the device by its GUID** (`value`) on future `req_acc` requests and will not ask the user for approval again ⁶. Instead, the handshake is auto-approved by the camera. In practice, when the same device reconnects:

- The app still sends the `cam.cgi?mode=accctrl&type=req_acc&value=<GUID>&value2=<Name>` request at the start of a session, but now the camera immediately sees the GUID in its authorized list.
- The camera then responds *almost instantly* with an "ok" (and the camera's ID info) without any user interaction needed. The LCD might briefly flash a connection message or directly show "Under Remote Control" without a prompt.

The stored pairing means **no prompt on the camera for future sessions**. As one guide notes: *"Once it has successfully connected it will store the connection and from thereon in will connect first time every time."* ⁶. This indicates that after the first handshake, the device will be allowed in automatically on subsequent uses, making the remote control connection much faster.

In terms of the protocol, the `accctl` command can be sent each time you initiate a new remote control session, and the camera will simply return `ok` immediately if the device is already known. (Some devices or apps may skip calling `accctl` on subsequent runs if they maintain a session, but typically each fresh Wi-Fi reconnection triggers the handshake again. The camera doesn't mind responding "ok" for a known device as a way to re-initiate the control session.)

It's worth noting that the camera keeps an **internal list of authorized device IDs** (Wi-Fi connection history). The capacity of this list is finite (for example, up to 16 devices on some models) – but it's usually enough for typical usage ⁸ ⁹. If you pair more devices beyond the limit, the camera will remove the oldest entry (or you may need to manually clear the list). In most Lumix cameras, you can review the Wi-Fi connection history in the menus (and on some, even label favorites) ¹⁰ ⁹. However, **the user interface usually does not offer a single-device removal** – you'd have to wipe all network settings to clear pairings, or just let new pairings overwrite old ones once the list is full ⁹. The absence of a prompt on subsequent connections is by design – it makes using the same phone or controller much more convenient after the first setup.

Example Handshake Request and Response

To illustrate a full handshake sequence, here's an example with a hypothetical device:

1. Device (App) -> Camera:

```
GET http://192.168.0.1/cam.cgi?mode=accctl&type=req_acc&value=12345678-90AB-CDEF-1234-567890ABCDEF&value2=John%20Phone
```

(This asks the camera for access. The GUID `1234...DEF` identifies the device and "John Phone" is the name.)

2. Camera behavior:

If `12345678-...-ABCDEF` is not recognized, the camera displays "Allow remote control from 'John Phone?'" on its screen (for example) and waits for the user to press OK. If the user approves, the camera internally marks this GUID as authorized.

3. Camera -> Device (response):

After approval, the camera sends back a response such as:

```
ok,DMC-GX880-1A2B3C,remote,encrypted
```

This indicates success. "DMC-GX880-1A2B3C" would be the camera's own ID (model and unique suffix) included for the app's reference. The `remote,encrypted` part confirms the mode (remote control started, and that streaming is encrypted).

4. Follow-up (camera state):

The camera LCD now shows it's under remote control, and the device can proceed to send other commands (like live view start, capture commands, etc.). On the device side, the app knows the handshake succeeded (it got "ok"), so it might then initiate live view streaming or other remote operations.

5. Error case (first time): If the user had *denied* the request on the camera, or no one answered, the camera would eventually return an error. In many cases, the camera simply does not return "ok", and the HTTP request might time out. Some cameras might return an error code like `err,####` (the

exact code is not well-documented in public reverse-engineering, since typically the official app just times out and shows a failure). In practice, a timeout or error means the device must retry pairing. As noted, repeating the process after ensuring the camera is in the correct Wi-Fi mode usually works on second attempt ⁶.

6. **Subsequent connection example:** If the device disconnects and later reconnects, it will send the same `accctl&...&value=12345678-...ABCDEF` request again. This time, the camera immediately responds with `ok,...` (with no user prompt). For instance:
`ok,DMC-GX880-1A2B3C,remote,encrypted`
(perhaps even faster, since the camera might not re-display any message except maybe a brief “Connected”.) The app can then proceed without delay.

These examples align with logs and tests shared by the community ² ⁷, demonstrating the request syntax and the “OK” response on success. In one community log, after the `accctl` and acceptance, the next step was to start the live view and send capture commands, which all succeeded once the camera was in remote mode ¹¹ ¹².

Revocation, Re-Pairing, and Device Management

Revocation: The `accctl` command itself does not provide a direct way to revoke a device’s access (there’s no “revoke” parameter in the HTTP API). To revoke or remove an authorized device, you must do it from the camera’s side (usually via the menu). Many Lumix models have a “Reset Network Settings” function that clears all Wi-Fi pairings/history ⁹. Some cameras also allow clearing the entire Wi-Fi history list, but not a specific entry (as evidenced by users struggling to delete individual saved connections on models like GH5 Mk II) ¹³. In short, you can **wipe all pairings** via a reset. After that, any device (even previously authorized ones) will be treated as new and will prompt again.

There isn’t an `accctl` sub-command for de-authorizing a device during normal operation. The camera assumes once authorized, it stays authorized until the user clears it in settings or the list capacity rolls over. So, *revocation is manual*: e.g., by deleting the Wi-Fi settings on the camera or performing a network reset ⁹.

Re-pairing: If, for some reason, you need to pair again (for example, the app was re-installed and generated a new GUID, or the camera’s pairing list was cleared), you would go through the `accctl` handshake fresh. The camera will treat a new GUID as a different device, even if the name is the same. There’s generally no problem having multiple GUIDs associated with the same phone or multiple phones – the camera will just have multiple entries in its authorized list. But note that if the camera has a limit (say 16 entries), adding a 17th will drop the oldest. If you *intentionally* want to re-pair a device (perhaps to change the name or reset the trust), you’d have to either reset the camera’s network settings or use up the slots. There’s no dedicated re-pair function; you simply do a new `req_acc` handshake and accept it on the camera, which either creates a new entry or updates an existing one if by chance the GUID was the same.

In practice, the official app keeps the same GUID so you normally don’t need to re-pair unless the camera was reset. If a user does need to “re-pair,” they usually delete the camera’s Wi-Fi profile on the phone and start from scratch on the camera as well (which accomplishes the same thing as a fresh handshake).

Multiple devices: The camera can pair with multiple different devices (one at a time for control, but stored for convenience). For example, you could pair your phone and your tablet on different occasions. Each will have its own GUID entry. When either connects, if recognized, it will be allowed in without prompt. If a **new** device tries later, the camera will prompt again. The previously paired ones remain remembered. As mentioned, the number of saved connections is limited (Panasonic documentation for some models suggests the history list is finite, and will start dropping oldest entries once full) ⁹ .

Spoofing and Security Considerations: Because the camera relies on the GUID to identify trusted devices, one security question is: what if a malicious device spoofs an existing GUID? In theory, if an attacker knows a previously authorized device's `value` (GUID), they could try to send an `accctl` request with that same GUID. The camera would see it as known and potentially allow access with no prompt, effectively “spoofing” the trusted device. However, several factors mitigate this risk:

- The GUID is a 128-bit value that is not easy to guess or brute-force. The attacker would need to know the exact GUID of a paired device. This GUID isn't broadcast openly; it's only sent during the handshake over the Wi-Fi connection. If the Wi-Fi network is secured (WPA2 encryption, which it is during direct camera-to-phone connection), intercepting that GUID would require breaking the Wi-Fi encryption or already being connected to the same network.
- In direct mode (camera as hotspot), an attacker would first have to join the camera's Wi-Fi (which itself has a password displayed on the camera for the user). So the attacker needs physical proximity and to see the Wi-Fi password or use some vulnerability. Only after connecting could they even attempt to send commands.
- If the camera is in infrastructure mode (connected through a router), and an attacker is on that same network, there is a bit more opportunity for sniffing or MITM. Still, they'd have to catch the GUID from a legitimate handshake or otherwise obtain it.

That said, it's **not known that Panasonic employs any challenge-response or rolling codes** beyond the GUID check. The protocol as observed is quite straightforward text HTTP commands. Therefore, if the GUID were compromised, someone could impersonate that device. The camera does not appear to use device-specific cryptographic authentication for commands – the “encrypted” in the response refers to liveview stream encryption, not command encryption. All command exchanges (like `accctl`, `camcmd`, `getsetting`, etc.) are plain HTTP queries ¹⁴ ¹⁵ . This means the system's security hinges on the Wi-Fi network security and the secrecy of the GUID.

In summary, spoofing a known device is theoretically possible but requires prior network access and knowledge of the GUID. There's no evidence of additional tokens or passwords for the older Image App protocol on models like GX880/GX80/GH5 – the handshake is the main gatekeeper. Users should still exercise caution: keep the camera's Wi-Fi password safe and reset pairings if a device is lost or stolen, to prevent that device (or its GUID) from being used by someone else.

For **revocation**, as noted, you'd do a network reset on the camera which wipes all authorized GUIDs, effectively forcing all devices to re-pair if they connect again ¹⁶ . There isn't a finer-grained method via the API or camera UI on most models.

Differences Across Lumix Models (GX880, GX80, GH5, etc.)

Panasonic's Wi-Fi protocol (often accessed via the Panasonic Image App or Lumix Sync App) is largely **consistent across the Lumix G/GX/GH series** in terms of command structure ¹⁷. The `accctrl` handshake process on the Lumix GX880 is essentially the same as on the GX80, GH5, and similar generation cameras. All these cameras use the `cam.cgi` interface with commands like `accctrl`, `getinfo`, `setsetting`, `camcmd`, etc., for remote operations ¹⁸ ¹⁹. Therefore, the behavior described above (prompt on first connection, then stored pairing) applies broadly to models like GX80 (GX85), GH5, G7, G9, etc., when using the Panasonic Image App protocol.

GX80 vs GX880: The GX880 (also known as GF10/GF90 in some markets) came a bit later than the GX80, but from a Wi-Fi standpoint, it uses the same Image App control protocol. Both lack Bluetooth LE, so they rely solely on the Wi-Fi handshake. Users have reported the exact same pairing steps on GX80 and GX880 – no notable differences. If anything, the GX880 being a newer model might have slightly updated firmware, but the command set and `accctrl` usage remain the same. (Panasonic did not overhaul the HTTP API for these minor model updates.)

GH5: The GH5 (2017) also uses the same protocol for the Image App. It does not have Bluetooth either (the GH5 Mark II added Bluetooth), so initial connection is via Wi-Fi and requires acceptance just like GX80/GX880. One minor difference: the GH5's Wi-Fi system can store more connection history entries (likely 16, similar to GH5 II's documentation) – but again, the mechanism is identical. From a developer's perspective, controlling a GH5 over Wi-Fi involves the same `accctrl` handshake and then issuing remote commands. In fact, community tools like the GMaster app and Lumix Link Desktop were developed against models like GH4/GX85 and work for GH5 as well ²⁰ – underscoring that the underlying protocol (including `accctrl`) did not change.

Notable exceptions or newer models:

- **Lumix GH5 Mark II / BGH1 / newer (Lumix Sync era):** Starting around 2020, Panasonic introduced the Lumix Sync app and cameras with Bluetooth. These cameras (e.g. GH5 II, S5, BGH1 box camera) can establish an initial pairing over Bluetooth which then automates the Wi-Fi connection. If Bluetooth pairing is used, the user confirmation on first Wi-Fi connect might not be required in the same way – the camera considers the Bluetooth pairing as authentication and will automatically switch to Wi-Fi. However, internally, the camera still uses an access control token for Wi-Fi. For example, the **DC-BGH1 (Lumix box camera)** can be controlled by Lumix Tether software over network. In that scenario, instead of a manual prompt (since BGH1 often runs headless), Panasonic uses an **authentication token (value3)** in the `accctrl` request. There is a variation `type=req_acc_a` (possibly "auto" or advanced) where a third parameter (`value3`) is required ²¹. This token is likely obtained from a prior pairing process (perhaps exchanged out-of-band or generated by Lumix Tether). One developer noted that to connect to a BGH1, you need a `value3` token, which can be sniffed from the official software traffic ²². Essentially, for these newer models, **the protocol extends `accctrl` with an extra authentication step**. The camera won't respond "ok" unless the correct token is provided, instead of a manual yes/no prompt. This is a key difference aimed at more seamless or remote operation without physical confirmation (since a BGH1 might be mounted in a hard-to-reach place).

- **Lumix S series and others with Lumix Sync:** These use a similar approach as the GH5 II – initial Bluetooth pairing to avoid Wi-Fi prompts. Once paired, Wi-Fi connects and the camera trusts the app. Under the hood they likely still do an `accctrl` or equivalent, but the user doesn't see a prompt. The older Image App based models (like GX880, GH5) didn't have this convenience; they always prompt on first connect via Wi-Fi.
- **Older models (GH3, early wifi cameras):** The GH3 and original LX100 (circa 2012-2014) used the old "Lumix Link" app. It's believed that even those early implementations had a similar handshake (possibly with slightly different parameter names or additional steps). By the time of GH4/GX8, Panasonic standardized on the `cam.cgi?mode=...` interface. So, GH3 might have differences, but GX80/GX880/GH5 are all in the modern family of the protocol.

In summary, **across GX80, GX880, GH5 (and contemporaries)**, the `accctrl` command's purpose and usage are the same. They all require the initial `req_acc` handshake with GUID and name, and store that pairing for future use without prompting. The main differences come with much newer models or different connection methods:

- Newer cameras with Bluetooth can skip the manual confirmation (using a prior pairing or token behind the scenes).
- Certain pro models (BGH1) use an authentication token (`value3`) in the `accctrl` request for headless operation ²².
- The number of devices remembered might vary (but generally the concept of remembering them is consistent).

However, these differences aside, the fundamental role of `accctrl` – to request access control – remains. Community-developed tools and reverse-engineering efforts have successfully used the same `accctrl` calls on many Lumix models ¹⁸ ¹⁴. If you are implementing control for a GX880, you can be confident that using `mode=accctrl&type=req_acc` with a GUID and name will initiate pairing just as it would on a GX80 or GH5.

Additional Notes

- **No explicit "unpair" command:** As noted, there is no documented HTTP command to undo a pairing (the camera doesn't expose that). Unpairing is done on the camera UI if needed.
- **One device at a time:** Even though multiple devices can be authorized, only one can control the camera at a given moment. If your camera is "Under Remote Control" by one device, another device cannot jump in concurrently. Usually, you must disconnect the first (or power cycle the camera) and then connect the second. The `accctrl` handshake will fail if the camera is already in a session with another device. In practice, the camera might return a busy or error if it's already paired live with someone else (or it might not even accept the Wi-Fi connection).
- **Live view encryption:** The "encrypted" part of the response means the camera's live view stream (usually a separate MJPEG or H.264 stream) is encrypted. Panasonic does this using a static key that the app knows. This is beyond `accctrl` itself, but it's why the response includes that word. It does not mean the HTTP commands are encrypted (they are not; they're plain text over the local network). It's specifically about the video stream. Tools like the HTML Image App project have had to implement the decryption to display live view ⁷ ²³.

- **Alternate connection modes:** If the camera is connected via an infrastructure Wi-Fi (to a router) rather than being its own hotspot, the handshake is the same – but ensure you use the right IP (the camera's IP on that network). The default IP `192.168.54.1` is common when the camera itself is the AP (Panasonic often uses 192.168.54.x). In infrastructure mode, the IP is assigned by your router (you'd have to find it). The `accctrl` process doesn't change, though.
- **Using community tools:** Projects like "lumixlink-desktop" and "python_lumix_control" implement this handshake in code. For instance, one library constructs the request as:

```
{"mode": "accctrl", "type": "req_acc", "value": device_guid, "value2": device_name}
```

as the payload ²⁴. This confirms the understanding of the parameters.

In conclusion, the `accctrl` command on the Lumix GX880 (and similar cameras) is the cornerstone of the Wi-Fi control authentication. Its request format requires a device ID and name, and its behavior encompasses user-approved pairing on first use and automatic recognition thereafter. Knowing how to properly use `accctrl` – and understanding the camera's response – is crucial for anyone attempting to reverse-engineer the Panasonic Image App protocol or create custom remote control applications for Lumix cameras. With the above details, one can reliably initiate and manage Wi-Fi remote connections to a GX880, GX80, GH5, etc., and be aware of how the camera handles access control under the hood.

Sources:

- Community forum posts and scripts demonstrating the `accctrl` request format and usage ¹ ².
- Reverse-engineering projects outlining Lumix Wi-Fi commands and pairing process ¹⁴ ⁷.
- Official Panasonic documentation implying the confirmation prompt on first connection and stored pairing afterwards ⁵ ⁶.
- Developer discussions on newer models requiring tokens for `accctrl` in lieu of user confirmation ²² ²¹.

¹ ⁴ ¹⁴ ¹⁵ ¹⁹ GitHub - cleverfox/lumixproto: Lumix GX80 wifi protocol reverse
<https://github.com/cleverfox/lumixproto>

² ³ Octolapse connect to camera but give Error - Webcams - OctoPrint Community Forum
<https://community.octoprint.org/t/octolapse-connect-to-camera-but-give-error/24605>

⁵ Image App | Panasonic Global
https://av.jpn.support.panasonic.com/support/spn/global/dsc/help/image_app/en/camera/DMC-TZ60/connect.html

⁶ ⁷ Would You Perhaps Be Interested In A Different GX80/85 Colour Profile??? - Page 4 - Cameras - EOSHD Forum
<https://www.eoshd.com/comments/topic/24995-would-you-perhaps-be-interested-in-a-different-gx8085-colour-profile/page/4/>

⁸ Connecting to a Smartphone
https://www.pavc.panasonic.co.jp/dscoi/DC-GH5M2/html/DC-GH5M2_DVQP2477_eng/0121.html

⁹ ¹⁰ ¹⁶ Connect to Wi-Fi Using Previously Saved Settings
https://www.pavc.panasonic.co.jp/dscoi/DC-BGH1/E/DC-BGH1_DVQP2279_eng/chapter13_05_03.htm

¹¹ ¹² Lumix GX80 wifi script works but gives error - Webcams - OctoPrint Community Forum
<https://community.octoprint.org/t/lumix-gx80-wifi-script-works-but-gives-error/33827>

13 Panasonic GH5 mkii - How to delete WiFi connection setting - Reddit

https://www.reddit.com/r/videography/comments/15slkix/panasonic_gh5_mkii_how_to_delete_wifi_connection/

17 18 Lumix HTTP/Wifi protocol · Issue #409 · gphoto/libgphoto2 · GitHub

<https://github.com/gphoto/libgphoto2/issues/409>

20 GitHub - Rambalac/GMaster: Panasonic Lumix Camera remote control application for Windows 10 (UWP)

<https://github.com/Rambalac/GMaster>

21 22 Panasonic Lumix DC-BGH1 Camera Control · Issue #402 - GitHub

<https://github.com/bitfocus/companion-module-requests/issues/402>

23 GitHub - auberginehill/panasonic-html-image-app: An unofficial and platform agnostic version of the Panasonic Image App for remote controlling a Lumix camera.

<https://github.com/auberginehill/panasonic-html-image-app>

24 modules/controller.py ... - GitLab

<https://git02.ncg.ingrid.pt/lex/imageapp/-/blob/f9de16ec3c191e4b264bc77993be90efbb795fbf/modules/controller.py>